PlushPal

MVP Detailed Implementation Plan

Execution roadmap for iPhone, Android, macOS, and Windows

Version: 1.1

Status: Execution baseline

Date: June 18, 2026

Target: Supervised cross-platform private beta

Companion: Product Requirements and Architecture Specification v5.0

Delivery decision

Eight gate-driven two-week sprints: feasibility, foundation, desktop vertical slice, local product, iOS, Android, connected features, and hardening.

Contents

1. Purpose and delivery outcome

This plan turns the approved PlushPal architecture into an executable delivery program. The first target is a supervised private beta that runs end to end on macOS, Windows, iPhone, and Android: parent onboarding, device assessment, local model installation, character and voice enrollment, local speech-to-text, age-controlled local conversation, optional experimental cloud conversation, local safety validation, local voice synthesis, encrypted storage, and complete local deletion.

The private beta is not a public child-directed store release. Public release requires separate provider contracts, child-privacy review, safety validation, store declarations, and release gates described in this plan.

1.1 MVP completion statement

The MVP is complete when a parent can configure one child age band and at least one character; a child can complete a stable ten-turn push-to-talk conversation with a capability-approved local model; all raw audio and voice assets remain local; conversation history follows the selected local retention policy; optional search and experimental cloud paths transmit only allowlisted text; and the complete workflow passes privacy, safety, interruption, recovery, and deletion tests on the supported device matrix.

1.2 Delivery principles

- Build one walking skeleton before broad feature work.
- Keep orchestration, policies, provider routing, and repositories in the shared Rust core.
- Keep platform SDK details in thin Swift, Kotlin, and desktop adapters.
- Keep C++ limited to speech and local-model inference boundaries.
- Make local mode the default and cloud mode an explicit parent choice.
- Treat safety, privacy, and deletion as product features with automated tests.
- Use release gates instead of calendar pressure to approve higher-risk stages.

2. Scope

2.1 In-scope MVP capabilities

- Shared Flutter interface for parent and child workflows.
- Parent PIN and platform authentication where available.
- Age-band policy selection for 4-5, 6-8, and 9-12.
- Device capability benchmark and signed model recommendation manifest.
- Downloadable Qwen3 1.7B Q4 baseline and Qwen3 4B Q4 capable-device tier, subject to benchmark approval.
- Local STT using the selected Sherpa-ONNX-compatible model.
- Local conversation inference through llama.cpp.
- Local zero-shot or reference-conditioned TTS through the selected ZipVoice-compatible deployment, subject to feasibility gate.
- Character creation, voice enrollment, preview, enable/disable, and deletion.
- Session-only history by default; optional encrypted local persistence.
- Local input screening, immutable age policy, structured output, local output screening, and safe fallbacks.
- Curated web lookup proof of concept for approved factual domains.

- Experimental OpenAI-compatible BYOK adapter for supervised development testing only.
- Signed desktop host serving embedded Flutter web assets over protected loopback.
- Native iOS and Android application hosts.
- Local redacted diagnostics and delete-all.

2.2 Explicitly deferred

- Public child-directed App Store and Play Store launch.
- PlushPal accounts, synchronization, hosted conversation gateway, subscriptions, and billing.
- Unrestricted web browsing and open-ended agents.
- Wake words, continuous listening, background capture, and smart-toy hardware connectivity.
- Multiple children, shared family accounts, remote parent dashboards, and cross-device history.
- Public voice export, voice sharing, character marketplace, and community content.
- Languages other than English.
- Guaranteed local LLM operation on every device.
- Automatic cloud analytics or crash-report uploads.

3. Architecture baseline

Layer	MVP implementation	Responsibility
Presentation	Flutter	Parent/child screens, accessibility, state rendering, local desktop web build
Application core	Rust	Use cases, session state, age policy, prompt compiler, repositories, provider routing, typed events
Local LLM	llama.cpp through C ABI	GGUF model loading, generation, cancellation, token accounting
Speech	C++/Sherpa-ONNX adapters	STT, audio normalization, speaker preparation, TTS
Mobile hosts	Swift and Kotlin	Permissions, audio sessions, lifecycle, Keychain/Keystore, native networking
Desktop host	Rust executable	Loopback gateway, embedded web assets, credential vault, database, native audio/inference
Storage	SQLCipher plus encrypted files	Profiles, settings, optional transcripts, voice assets, model manifests
Cloud adapter	Rust/native HTTPS	Experimental stateless text-only provider calls
Curated search	Rust/native HTTPS	Sanitized queries, bounded evidence, untrusted-content handling

3.1 Dependency rule

Flutter SHALL never access SQL, model files, provider keys, or voice assets directly. The shared core SHALL depend on interfaces for storage, inference, networking, key management, and clocks. Platform adapters SHALL implement those interfaces. Local providers receive bounded internal context; cloud and search adapters receive separate allowlisted DTOs and SHALL not query repositories directly.

3.2 Runtime sequence

```
Flutter command
  -> platform bridge
  -> Rust use case
  -> audio/STT adapter
  -> local input policy
  -> local or experimental cloud conversation provider
  -> structured response validator
  -> local output policy
  -> local TTS
  -> typed progress/playback events
  -> Flutter state
```

4. Team and planning assumptions

The baseline plan assumes the following effective capacity:

Role	Allocation	Primary ownership
Technical lead / principal engineer	1.0	Architecture, interfaces, reviews, release gates
Rust and inference engineer	1.0	Core, llama.cpp, speech ABI, desktop host
Flutter/mobile engineer A	1.0	Shared UI, iOS host, accessibility
Flutter/mobile engineer B	1.0	Shared UI, Android host, model manager
QA/automation engineer	1.0	Device automation, privacy tests, regression suites
Product designer/researcher	0.5	Parent onboarding, child interaction, usability
Safety/privacy advisor	0.25	Policy corpus, disclosures, retention and provider review

With fewer than four full-time engineers, preserve the milestone order and reduce simultaneous platform work rather than weakening architecture, safety, or test scope. All estimates are planning ranges, not commitments, and shall be recalibrated after the feasibility gate.

5. Delivery roadmap

The plan uses eight two-week sprints for the private-beta target. The calendar may change after model benchmarks; exit criteria do not.

Sprint	Goal	Demonstrable outcome	Exit gate
0	Feasibility and decisions	Benchmarked STT, local LLM, TTS, and ABI prototypes	G0 feasibility
1	Repository and core foundation	CI, core state machine, generated interfaces, encrypted test repository	G1 foundation
2	Desktop walking skeleton	Record -> STT -> local LLM -> standard TTS -> playback	G2 vertical slice
3	Voice, policy, and model management	Voice enrollment, cloned preview, age policies, model installer	G3 local MVP
4	iOS integration	Complete supervised workflow on reference iPhones	G4 iOS
5	Android integration	Complete supervised workflow on reference Android devices	G5 Android
6	Cloud experiment and curated search	Stateless BYOK testing, local history, bounded web evidence	G6 feature complete
7	Hardening and private beta	Security, privacy, safety, accessibility, recovery, installers	G7 private beta

Public store release begins only after G7 and adds provider contracting, independent review, store compliance, and expanded beta evidence.

6. Sprint 0: feasibility and architecture decisions

6.1 Objectives

Prove the three highest-risk capabilities before building product surfaces: child-speech STT, useful local conversation, and acceptable voice-cloning TTS on representative hardware. Prove that Rust, C++, Flutter, iOS, Android, and desktop can exchange asynchronous jobs safely.

6.2 Work items

SPIKE-INF-001: Device matrix. Acquire or nominate at least two iPhones, two Android phones, one Apple Silicon Mac, and one Windows x64 computer. Record OS, memory class, storage, CPU/GPU/NPU, and thermal behavior.

SPIKE-INF-002: Local LLM benchmark. Convert or obtain verified GGUF Q4 builds of candidate 1.7B and 4B models. Measure download size, load time, first-token latency, tokens per second, peak memory, 10-turn stability, and output quality in short child-dialogue prompts.

SPIKE-STT-001: STT benchmark. Evaluate at least one compact English STT model using child and adult test recordings across quiet, normal-home, and moderate-noise conditions. Measure word error rate, end-of-speech latency, memory, and recovery after interruption.

SPIKE-TTS-001: Voice-cloning benchmark. Validate the selected zero-shot TTS engine with 15-, 20-, and 30-second authorized reference samples. Measure speaker similarity, intelligibility, synthesis real-time factor, peak memory, and failure behavior.

SPIKE-ABI-001: Native boundary. Build a minimal Rust host calling a C ABI for a cancellable inference job and emitting progress events. Verify AddressSanitizer and thread-safety behavior on desktop.

SPIKE-MOB-001: Flutter/native bridge. Prove one asynchronous native job, cancellation, progress, and buffer transfer on iOS and Android without blocking the UI thread.

SPIKE-LIC-001: License inventory. Record source, model license, redistribution rights, commercial-use restrictions, attribution, checksum, and version for every candidate runtime and weight file.

6.3 Deliverables

- Benchmark report and raw result schema.
- Approved model shortlist and minimum-device proposal.
- C ABI v0 prototype.
- ADR-001 through ADR-006 covering shared core, UI, speech runtime, local LLM runtime, storage, and desktop delivery.
- Updated risk register and estimates.

6.4 G0 acceptance criteria

- At least one STT configuration meets the provisional accuracy and latency targets on minimum devices.
- At least one local model produces coherent three-sentence dialogue at acceptable latency on a standard phone.
- Voice cloning works acceptably on desktop and at least one target phone, or a documented fallback plan is approved.
- No unresolved license blocker exists for the selected private-beta distribution.
- The native job prototype cancels cleanly and passes memory-safety instrumentation.

If TTS cannot meet mobile targets, the MVP SHALL use a standard local TTS voice on mobile while retaining cloned voice on desktop until a replacement model passes. The architecture must not be rewritten around one TTS engine.

7. Sprint 1: foundation

7.1 Repository bootstrap

Create the monorepo:

```
plushpal/
  apps/flutter_ui/
```

```
apps/ios_host/
apps/android_host/
apps/desktop_host/
crates/core_domain/
crates/application/
crates/policy_engine/
crates/provider_api/
crates/local_llm_llamacpp/
crates/search_api/
crates/curated_search/
crates/encrypted_storage/
crates/desktop_gateway/
native/inference_abi/
native/speech_engine/
schemas/events/
schemas/local_api/
schemas/search_evidence/
models/manifests/
tests/privacy/
tests/safety/
tests/e2e/
docs/adr/
```

7.2 Work items

FOUND-REP-001: Pin Flutter, Dart, Rust, CMake, compilers, ONNX Runtime, llama.cpp, and code-generation versions. Add reproducible bootstrap scripts.

FOUND-CI-001: Add formatting, linting, unit tests, desktop debug builds, dependency/license scanning, secret scanning, and artifact retention. Mobile simulator builds become required before Sprint 4.

FOUND-SCHEMA-001: Define versioned command, event, provider-response, error, and model-manifest schemas. Generate Dart/Rust/native bindings where practical.

FOUND-ELIG-001: Define a signed, expiring provider-eligibility registry keyed by provider, model, region, age range, retention requirement, and release channel. The runtime fails closed when eligibility is missing, expired, or incompatible.

CORE-STATE-001: Implement the conversation state machine: Idle, Capturing, Transcribing, ScreeningInput, Generating, ScreeningOutput, Synthesizing, Playing, Cancelled, and Failed.

CORE-JOB-001: Implement job IDs, deadlines, cancellation tokens, typed progress events, stale-result rejection, and one-active-turn-per-session behavior.

CORE-ERR-001: Define normalized errors for permission, model, storage, provider, policy, timeout, resource pressure, cancellation, and internal failures.

STORE-001: Implement encrypted repository interfaces, schema migrations, transactional writes, and in-memory test adapters.

KEY-001: Define the secret-reference and wrapped-key interfaces. Add desktop development implementation without storing real secrets in source or fixtures.

7.3 G1 acceptance criteria

- A fresh checkout can build and test through documented commands.
- CI rejects formatting, lint, test, secret, and license failures.
- The core state machine has exhaustive transition tests.

- Cancellation and stale response tests pass under randomized timing.
- Database migration v1 and encrypted repository round-trip tests pass.

8. Sprint 2: desktop walking skeleton

8.1 Desktop host

DSK-HOST-001: Build the Rust desktop executable that serves embedded Flutter web assets, binds only to loopback, selects an ephemeral port, creates a launch secret, and opens the default browser.

DSK-SEC-001: Implement Host and Origin validation, strict CORS, restrictive CSP, authenticated WebSocket upgrade, payload caps, idle shutdown, and DNS-rebinding tests.

DSK-AUD-001: Implement microphone selection, capture, VAD boundary, playback, cancellation, and audio-route error handling.

8.2 Walking-skeleton conversation

INF-STT-001: Wrap the approved STT engine behind `SpeechToText` with model load, utterance transcription, progress, confidence where supported, and cancellation.

INF-LLM-001: Wrap llama.cpp behind `ConversationProvider`, including model load, token streaming to the core, sampling configuration, context cap, deadline, and cancellation.

INF-TTS-001: Use standard local TTS temporarily if cloned TTS is not ready. Return typed PCM/audio frames through the playback adapter.

UI-CONV-001: Implement a developer conversation screen showing selected character, state, push-to-talk, cancel, progress, non-sensitive errors, and playback.

E2E-DSK-001: Automate the deterministic path using fixture audio and a deterministic model/provider test double; maintain a manual physical-audio test checklist.

8.3 G2 acceptance criteria

- Signed-development desktop host starts without Node or npm.
- Browser UI cannot access the service without the launch session.
- Fixture and live microphone audio complete STT -> LLM -> TTS -> playback.
- A turn can be cancelled during every asynchronous stage.
- Ten sequential turns complete without stale playback, resource leakage, or corrupted state.

9. Sprint 3: local product MVP

9.1 Parent onboarding

UI-ONB-001: Build disclosure, parent PIN, age-band, device assessment, model recommendation, download acknowledgement, and completion screens.

POL-AGE-001: Implement immutable versioned age policies for 4-5, 6-8, and 9-12. Policies control sentence/character limits, vocabulary guidance, search access, prohibited behavior, personal-data handling, and trusted-adult escalation.

MODEL-MGR-001: Implement signed manifest retrieval, local bundled fallback manifest, free-space check, resumable download where supported, checksum/signature verification, staging, atomic activation, rollback, and uninstall.

MODEL-BENCH-001: Run a bounded local benchmark and recommend `lite`, `standard`, `enhanced`, or `unsupported`. Persist only capability classification and non-sensitive benchmark results.

9.2 Character and voice

CHR-001: Implement character create/edit/enable/disable/delete with approved personality vocabulary and parent guidance limits.

VOICE-001: Implement WAV/common-audio import, local transcode, duration/noise/clipping validation, authorization attestation, local speaker preparation, encrypted storage, and preview.

VOICE-DEL-001: Implement character crypto-erasure, cache cleanup, database cascade, interrupted-deletion recovery, and verification tests.

9.3 Safety pipeline

SAFE-IN-001: Implement local PII patterns, unsupported-language detection, high-confidence block/escalation categories, and input length limits.

SAFE-PROMPT-001: Implement versioned system instructions, age policy, character fields, untrusted transcript delimiters, bounded context, and structured response schema.

SAFE-OUT-001: Implement strict JSON parsing, sentence/character limits, URL/contact detection, prohibited request patterns, markup stripping, and safe local fallbacks.

SAFE-CORPUS-001: Create the initial automated corpus for prompt injection, secrecy, contact exchange, dangerous activity, sexual content, graphic violence, manipulation, distress, abuse disclosure, benign near-matches, and age appropriateness.

9.4 G3 acceptance criteria

- A parent can complete onboarding and install a recommended model.
- A parent can enroll and delete a voice without network transmission.
- Local conversations apply the selected age policy and produce valid structured responses.
- Every failed policy stage results in an approved local fallback and no TTS of rejected content.
- Delete-character and delete-all pass after normal, cancelled, and crash-recovery scenarios.

10. Sprint 4: iOS integration

10.1 Work items

IOS-BRIDGE-001: Package Rust and C++ artifacts for device and simulator development, expose the versioned ABI, and implement generated Flutter plugin calls.

IOS-KEY-001: Implement Keychain-wrapped installation keys and secret references. Verify that API keys never return to Dart.

IOS-STORE-001: Configure application support directories, file-protection classes, backup exclusions, SQLCipher, model storage, and migration recovery.

IOS-AUD-001: Implement `AVAudioSession` and `AVAudioEngine` capture/playback, permission timing, interruption, route changes, Bluetooth policy, calls, headphones, device lock, and cancellation.

IOS-MEM-001: Implement memory-pressure events and model eviction. Benchmark sequential and resident STT/LLM/TTS strategies.

IOS-UI-001: Complete parent and child flows, dynamic text, VoiceOver labels, focus order, reduced motion, and touch targets.

IOS-E2E-001: Run automated state tests and physical-device end-to-end tests on minimum and reference iPhones.

10.2 G4 acceptance criteria

- Onboarding through ten-turn playback completes on both iPhone classes.
- Calls, route changes, backgrounding, lock, and memory pressure do not corrupt state or retain recording.
- Keychain, backup exclusion, encrypted storage, and deletion inspections pass.
- VoiceOver and dynamic-text critical flows pass manual accessibility review.

11. Sprint 5: Android integration

11.1 Work items

AND-BRIDGE-001: Package arm64-v8a native libraries, implement the generated Flutter plugin, and validate ABI/version mismatch handling.

AND-KEY-001: Implement Android Keystore key wrapping and application-internal protected storage.

AND-AUD-001: Implement `AudioRecord`, `AudioTrack`, audio focus, permission timing, route changes, Bluetooth, calls, backgrounding, and cancellation.

AND-MODEL-001: Integrate the approved model-delivery mechanism for private beta and preserve an abstraction for Play Asset Delivery or first-party CDN release.

AND-MEM-001: Implement low-memory and thermal degradation, model eviction, and supported-device capability gating.

AND-UI-001: Complete TalkBack, font scaling, navigation, focus, touch-target, and system-back behavior.

AND-E2E-001: Run physical-device end-to-end tests on minimum and reference Android devices.

11.2 G5 acceptance criteria

- The complete local workflow passes on both Android classes.
- Permission denial, audio focus, interruptions, low memory, and thermal events fail safely.
- Keystore, storage, network capture, backup behavior, and deletion inspections pass.
- Unsupported devices are rejected before model download with an understandable alternative.

12. Sprint 6: cloud experiment, history, and curated search

12.1 Experimental cloud adapter

CLOUD-API-001: Implement an OpenAI-compatible adapter using stateless generation, `store: false` where supported, no remote conversation IDs, structured output, deadline, cancellation, and normalized errors.

CLOUD-ELIG-001: Enforce the provider-eligibility registry before credential validation and every cloud turn. Parent consent or a valid API key SHALL NOT bypass provider/model/region eligibility.

CLOUD-KEY-001: Add parent-only credential entry, direct vault storage, validation, removal, and log/snapshot redaction tests.

CLOUD-MIN-001: Implement a request allowlist containing only policy version, age band, character alias/traits, bounded recent turns, local summary, current transcript, and output constraints.

CLOUD-DISC-001: Add supervised-prototype disclosures explaining external text processing and provider retention. Do not label the experimental adapter suitable for public child use.

CLOUD-POLICY-001: Ship the generic OpenAI-compatible adapter only in development/private-beta channels. OpenAI production use requires an approved under-18 arrangement with the required retention controls; Gemini Developer API remains disabled unless its then-current terms and an approved arrangement satisfy the target child-directed use.

12.2 Local history

HIST-001: Implement session-only default, optional encrypted retention, expiry cleanup, parent review, delete session, and delete all.

HIST-SUM-001: Implement bounded local context and local summary generation. Local encrypted history is authoritative; cloud providers receive a separately minimized projection and no remote conversation ID.

12.3 Curated search proof of concept

SEARCH-001: Define `SearchProvider`, source allowlist, SafeSearch requirement, query sanitizer, PII removal, timeout, result cap, and local cache TTL.

SEARCH-FETCH-001: Implement HTTPS-only fetch with private-address blocking, redirect validation, size/type limits, script/style removal, visible-text extraction, and untrusted-evidence labeling.

SEARCH-ANSWER-001: Require grounded responses with source IDs, block unsupported claims where evidence is insufficient, and expose sources in parent mode.

12.4 G6 acceptance criteria

- Network capture confirms cloud requests contain only allowlisted text fields.
- Missing, expired, or incompatible provider eligibility prevents cloud activation and every cloud turn.
- Credentials never appear in Dart state, logs, database, URLs, or diagnostics.
- Removing a cloud credential leaves local mode functional.
- Local history expiry and deletion pass simulated-clock and restart tests.
- Search cannot reach loopback/private networks or invoke arbitrary tools.

- Retrieved prompt injection text cannot change tool permissions or policy.

13. Sprint 7: hardening and private beta

13.1 Quality and security

QA-E2E-001: Run the complete matrix on all supported physical devices and desktop systems.

QA-PERF-001: Record cold start, model load, first token, STT/TTS real-time factor, end-to-end latency, peak memory, battery, thermal state, disk use, and 30-turn stability.

QA-PRIV-001: Capture network traffic for onboarding, enrollment, local conversation, cloud conversation, search, deletion, and diagnostics. Verify field allowlists.

QA-SAFE-001: Run the full safety corpus by age band and model/provider. Record unsafe pass rate, over-block rate, malformed output, and fallback quality.

SEC-DSK-001: Test desktop session theft, CSRF, malicious origins, DNS rebinding, WebSocket abuse, oversized payloads, path traversal, and localhost port scanning.

SEC-NATIVE-001: Run dependency, secret, binary-hardening, exported-component, URL-scheme, backup, and filesystem reviews.

A11Y-001: Complete VoiceOver, TalkBack, keyboard, contrast, dynamic text, reduced motion, and child-flow usability reviews.

REL-001: Produce signed/notarized private-beta artifacts, checksums, SBOMs, model/license notices, installation guides, privacy disclosure, test-data instructions, support runbook, and rollback procedure.

13.2 G7 acceptance criteria

- No open critical or high-severity security/privacy finding.
- Safety thresholds approved for supervised beta; known limitations documented.
- All mandatory requirements map to passing tests or approved time-bound waivers.
- Installation, upgrade, rollback, data migration, uninstall, and delete-all pass.
- A supervised pilot can be supported without collecting conversation content.

14. Cross-cutting engineering workstreams

14.1 Core and API discipline

- Version every ABI structure with size and version fields.
- Use opaque handles, fixed-width integers, explicit buffer ownership, typed error codes, and explicit free functions.
- Never allow `std::string`, `std::vector`, Rust layout types, exceptions, or panics to cross an ABI.
- Keep all long operations asynchronous and cancellable.
- Record ADRs whenever a platform exception changes shared behavior.

14.2 Model lifecycle

- Maintain signed manifests in source control and release storage.

- Pin exact weight hashes and licenses.
- Stage and verify downloads before activation.
- Preserve one last-known-good version.
- Reject engine/model compatibility mismatches.
- Keep model files separate from sensitive user encryption domains; integrity, not secrecy, is their primary requirement.

14.3 Privacy engineering

- Maintain a machine-readable network DTO allowlist.
- Add unit tests proving sensitive entities cannot serialize into provider/search requests.
- Default to session-only history.
- Prevent sensitive data in logs, crash messages, screenshots, filenames, metrics, and diagnostics.
- Verify backup behavior and crypto-erasure on each platform.

14.4 Safety engineering

- Version policies, prompts, filters, and corpus together.
- Evaluate every model/policy combination before activation.
- Keep deterministic fallbacks independent of model availability.
- Do not let parent guidance override immutable safety instructions.
- Treat web content and child transcripts as untrusted data.

15. Test architecture

Test layer	Scope	Required in CI
Unit	Domain rules, policies, state transitions, prompt compilation, retention	Every change
Property/fuzz	Parsers, ABI buffers, Unicode names, JSON, URLs, audio metadata	Nightly and release
Contract	Provider adapters, schemas, model manifests, desktop API	Every change
Integration	SQLCipher, key vaults, inference adapters, migrations, cancellation	Every merge where supported
Desktop E2E	Loopback auth and deterministic conversation	Every merge
Mobile simulator	Navigation, state, bridge error behavior	Every merge after Sprint 3
Physical device	Audio, inference, interruptions, memory, thermal	Nightly lab/release
Safety	Age/model/provider corpus	Every policy/model change
Privacy	Network allowlist, storage, backup, deletion, diagnostics	Release and weekly
Security	Static analysis, dependencies, desktop/native	Every release candidate

Test layer	Scope	Required in CI
	attack cases	

15.1 Test data rules

Use synthetic text, synthetic/generated audio, or recordings from consenting adult team members. Never put real child conversations, names, voice samples, provider keys, or production endpoints in source, fixtures, CI artifacts, screenshots, or bug reports.

16. First sprint-ready backlog

These tickets can be created immediately, before Sprint 0 begins:

Priority	Ticket	Owner profile	Dependency	Acceptance evidence
P0	SPIKE-INF-001 device matrix	Tech lead	None	Approved matrix and measurement script
P0	SPIKE-INF-002 Qwen benchmark	Inference	Device matrix	JSON/CSV benchmark results and recommendation
P0	SPIKE-STT-001 child-speech evaluation	Inference + QA	Test corpus	Accuracy/latency report
P0	SPIKE-TTS-001 voice-cloning evaluation	Inference	Authorized samples	Quality/performance report
P0	SPIKE-ABI-001 cancellable native job	Rust/inference	None	ASan-tested demo
P0	SPIKE-MOB-001 Flutter bridge demo	Mobile	ABI draft	iOS and Android progress/cancel demo
P0	SPIKE-LIC-001 license ledger	Lead/advisor	Candidate list	Approved license table
P1	FOUND-REP-001 monorepo bootstrap	Tech lead	ADR drafts	Reproducible local build
P1	FOUND-CI-001 CI skeleton	QA/lead	Repo	Green build/lint/test workflows
P1	SAFE-CORPUS-001 corpus taxonomy	Safety + QA	Age bands	Versioned initial corpus

17. First ten working days

Days 1-2

- Confirm team roles and decision owners.
- Create the repository, issue tracker, ADR template, risk register, and license ledger.
- Freeze the initial device matrix and acquire missing test devices.
- Define benchmark result schemas and synthetic/authorized test-data rules.

Days 3-5

- Build desktop llama.cpp benchmark harness.
- Build STT and TTS command-line benchmark harnesses.
- Create the first C ABI job/cancellation prototype.
- Draft age-policy schema and safety-corpus categories.
- Stand up formatting, lint, unit-test, secret-scan, and dependency-scan CI.

Days 6-8

- Run benchmarks on desktop and mobile reference devices.
- Prove Flutter-to-native asynchronous progress and cancellation.
- Review memory and thermal results; select initial model candidates.
- Record licensing and redistribution findings.

Days 9-10

- Hold G0 review.
- Approve or revise minimum devices and model tiers.
- Finalize initial ADRs and Sprint 1 backlog.
- Publish the benchmark report and revised effort forecast.

18. Definition of ready and definition of done

18.1 Story ready

A story is ready when it has an owner, user or system outcome, dependencies, interface/schema impact, privacy and safety classification, acceptance tests, representative target devices, and no unresolved product decision that changes its implementation.

18.2 Story done

A story is done when code and tests are merged; relevant platforms build; error, cancellation, accessibility, privacy, and logging behavior are handled; schemas and ADRs are updated; no secret or sensitive fixture is introduced; and acceptance evidence is attached.

18.3 Release candidate done

A release candidate is done when mandatory requirements are traced, artifacts and models are reproducible and signed, SBOM/license notices exist, migrations and rollback pass, critical device flows pass, safety/privacy/security gates pass, and known limitations are approved and disclosed.

19. Decision and escalation ownership

Decision	Accountable owner	Required reviewers
Minimum device and model tiers	Technical lead	Inference, mobile, product
Model/license approval	Product owner	Technical lead, legal/privacy
Age policy and safety thresholds	Product/safety owner	QA, privacy, engineering
Cloud provider eligibility	Product owner	Legal/privacy, security, technical lead
Storage and retention defaults	Privacy owner	Product, security, mobile
ABI/schema compatibility	Technical lead	All platform owners
Beta go/no-go	Product owner	Technical lead, QA, safety/privacy

Block release when a decision affects child safety, provider terms, biometric use, data retention, external transmission, or unsupported-device behavior and has no accountable approval.

20. Risks and contingency triggers

Risk	Trigger	Immediate contingency
Mobile voice cloning exceeds budget	TTS slower than 1.5x real time or causes termination	Use standard local TTS on affected tier; keep replaceable interface
Local 1.7B quality insufficient	Dialogue/safety evaluation misses threshold	Use 4B capable tier; limit local mode; supervised cloud experiment
Model combination exceeds memory	Repeated pressure/termination during ten turns	Sequential load/eviction, smaller STT/TTS, lower context, device exclusion
Child-speech STT inaccurate	WER above approved threshold	Push-to-talk UX, retry/confirmation, alternate model, narrower device/language scope
Desktop browser gateway vulnerable	Any high-severity local attack finding	Suspend desktop beta until fixed and retested
Cloud terms	Provider/model/region cannot satisfy child-use	Fail closed; keep adapter development-only;

Risk	Trigger	Immediate contingency
unsuitable or eligibility expires	and retention requirements	ship local supervised beta
Schedule pressure threatens safety/privacy tests	Required corpus/privacy tests incomplete	Reduce platform/features; do not waive mandatory gates

21. Path from private beta to public release

After G7, the public-release program must add:

- Qualified legal review for COPPA, biometric/voice laws, regional child-privacy rules, and provider terms.
- A production cloud decision: local-only release or a provider/model/region arrangement approved for the target age range and required retention controls, such as an approved zero-data-retention path.
- Independent application-security and desktop-gateway review.
- Expanded supervised family beta with documented consent and support procedures.
- Store privacy/data-safety declarations, parental-gate review, model download rules, and child-category compliance.
- Incident response, vulnerability intake, model/policy rollback, support, and deletion runbooks.
- Final production device/OS matrix and published minimum storage requirements.

Public release SHALL not inherit the experimental parent-BYOK cloud option without separate approval.

22. Immediate start checklist

- Name the product owner, technical lead, platform owners, QA owner, and safety/privacy reviewer.
- Create the monorepo and issue tracker from the structure in this plan.
- Acquire the Sprint 0 device matrix.
- Download candidate model weights only from approved sources and record hashes/licenses.
- Create synthetic and consenting-adult evaluation data.
- Open the ten first-sprint tickets.
- Schedule the G0 review for the end of the feasibility sprint.
- Begin with benchmark harnesses and the asynchronous ABI prototype, not production UI screens.

Appendix A: Required ADRs

1. Shared Rust application core and stable ABI.
2. Flutter presentation and native host boundaries.
3. Local LLM runtime and model tiers.
4. STT and TTS engines and fallback strategy.
5. Encrypted storage, key management, backup exclusion, and deletion.
6. Desktop loopback authentication and browser delivery.
7. Age-policy schema and safety pipeline.
8. Local history and context minimization.
9. Experimental cloud adapter boundaries.

10. Curated web lookup and untrusted-content handling.
11. Production provider-eligibility registry, approval ownership, expiry, and fail-closed behavior.

Appendix B: Core engineering commands to define

The repository bootstrap SHALL provide stable commands equivalent to:

```
make bootstrap       install/check pinned toolchains
make format          format Dart, Rust, C++, Swift/Kotlin glue
make lint            run static analysis
make test            run fast unit and contract tests
make test-safety     run age-policy regression corpus
make test-privacy    run serialization/network allowlist tests
make build-desktop   build signed-development desktop host and UI assets
make build-ios       build iOS simulator/device development artifacts
make build-android   build Android development artifact
make benchmark       run selected inference benchmark suite
make sbom            generate software/model bill of materials
```

The actual implementation MAY use another task runner, but one documented entry point must orchestrate each workflow consistently in local development and CI.

Appendix C: Private-beta evidence package

The G7 review package SHALL contain:

- Requirements-to-test traceability export.
- Device and inference benchmark report.
- Safety evaluation report by age band and model/provider.
- Privacy network-capture and storage-inspection report.
- Security review and resolved findings.
- Accessibility checklist and known limitations.
- Model/runtime license ledger and SBOM.
- Signed artifact checksums and rollback instructions.
- Data deletion and retention verification.
- Supervised-beta disclosure, consent, and support materials.

Appendix D: Version 1.1 change summary

- Aligns execution with the v5.0 local-first architecture and makes a capability-approved llama.cpp model the MVP completion path.
- Adds the signed provider-eligibility registry and separates private-beta BYOK experimentation from production cloud approval.
- Makes local encrypted history authoritative and gives external adapters separate minimized DTOs.
- Expands curated-search work to enforce sanitized queries, bounded untrusted evidence, and private-network blocking.
- Adds provider eligibility and external-mode boundaries to acceptance evidence and release decisions.